

PhishGuard Omni: Unified Autonomous Threat Defense

Comprehensive Hackathon Execution Blueprint

Chirag Soni

B.Tech CSE (AI & ML), KCC Institute of Technology and Management

Hack Celestia 2026

1 Executive Summary & Problem Statement

Students, employees, and everyday users frequently receive suspicious emails, SMS messages, and URLs, but often cannot tell exactly what makes a message dangerous[cite: 1]. A binary "safe/unsafe" label is not enough because users need understandable evidence before deciding what to do[cite: 1]. Furthermore, phishing attacks are increasingly combined with SMS and call bombing to induce Multi-Factor Authentication (MFA) fatigue[cite: 3].

PhishGuard Omni solves this by providing a layered, explainable detection interface. When a user pastes a message, email body, or URL, the system extracts indicators (urgency, impersonation, mismatched links) and produces a risk level, evidence cards, and safe next actions[cite: 1]. It also features an automated infrastructure defense layer that scrubs volumetric application floods and SMS bombing scripts before they breach the perimeter.

2 Threat Taxonomy & Mitigations

2.1 Explainable Phishing & Quishing (QR Phishing)

- **Typosquatting & Subdomain Spoofing:** Attackers use misspelled domains (e.g., `paypal1.com`) or prepend trusted brands to malicious root domains (e.g., `yourbank.com.malicious-` [cite: 3]).
- **Quishing:** Malicious QR codes route a scanned phone straight to a credential-harvesting page, bypassing standard email and URL filters[cite: 3].
- **Smishing:** Phishing via SMS using fake delivery notices or bank alerts containing malicious links[cite: 3].

2.2 Telephony Abuse (SMS & Call Bombing)

- **MFA Fatigue / Push-Bombing:** Repeated MFA push notifications or OTP SMS messages are sent until the victim approves one out of annoyance or confusion[cite: 3].

- **Mitigation:** The system tracks cross-platform OTP request velocity. If a threshold is breached, it automatically rate-limits the IP address and issues a lockdown alert.

2.3 Application Denial of Service (DDoS)

- **Mitigation:** Redis-backed Token Bucket rate limiting is applied at the API edge to intercept Layer 7 HTTP floods and fast-cycling proxy attacks.

3 System Architecture & Tech Stack

The platform is built using a modern, scalable stack: React or Next.js; FastAPI; Python; URL parsing; regex/rules; optional transformer/LLM; SQLite/PostgreSQL[cite: 2].

- **Layer 0 (Ingestion & Scrubbing):** FastAPI middleware running Redis rate-limiting to drop DDoS and SMS-bombing traffic.
- **Layer 1 (Deterministic Rules):** Python pipeline running `tlextract` for domain parsing, regex for risk rules, and Pyzbar for QR screenshot analysis[cite: 1, 2].
- **Layer 2 (Feature Extraction):** NLP features extract psychological urgency and credential requests[cite: 1].
- **Layer 3 (Explainability & SOAR):** An LLM explanation layer that keeps evidence separate from generated explanations, showing the UI exactly which signals triggered the warning[cite: 1].

4 Autonomous SOAR Workflows

Strong demo value comes from a visible security workflow and a clear user action after detection[cite: 2].

1. **Calculate Risk Score:** Classifies threats into Low/Medium/High bands based on extracted indicators[cite: 1].
2. **Generate Evidence Cards:** Displays 5–10 explainable indicators per threat[cite: 1].
3. **Automated Abuse Reporting:** Automatically compiles incident logs and formats them for immediate dispatch to CERT-In (for Indian infrastructure defense), the 14C National Cyber Crime Reporting Portal (cybercrime.gov.in), or the global Anti-Phishing Working Group (APWG)[cite: 3].
4. **Safe-Action Checklist:** Recommends immediate remediation steps to the user (e.g., calling the 1930 helpline for financial fraud)[cite: 1, 3].

5 Engineering Challenges & Solutions

6 48-Hour Build Roadmap

- **Phase 1: MVP Foundation (Hours 0-16):** Input email/SMS/URL[cite: 1]. Normalize and extract links[cite: 1]. Build Python/FastAPI backend with Redis rate-

Challenge	Solution
LLM Hallucinations	Use the LLM only to format explanations, not to make detection decisions. Run deterministic rules first[cite: 1].
False Positives	Implement feedback-based threshold tuning and clearly label the system as decision support, not perfect detection[cite: 2].
Proxy Rotation	Track API abuse by both source IP and target phone number destination using Redis sliding windows.

limiting.

- **Phase 2: Detection Engine (Hours 16-32):** Run URL and text risk rules[cite: 1]. Calculate risk score[cite: 1]. Build QR/link screenshot analysis[cite: 2].
- **Phase 3: Explainability & UI (Hours 32-48):** Generate feature-level evidence[cite: 1]. Show explanation and recommended safe action[cite: 1]. Build React/Next.js dashboard.

7 The Live Demo Plan

During the hackathon pitch, demonstrate measurable precision and explainability by showing three specific messages[cite: 2]:

1. **A normal college notice:** Proves the system does not produce false positives on benign links[cite: 2].
2. **A fake bank alert:** Triggers the SMS-bombing/urgency NLP rules and outputs an automated CERT-In report[cite: 2, 3].
3. **A fake scholarship message:** Compare the evidence cards and demonstrate that the system explains *why* the risk changed[cite: 2].

8 Core Code Foundations

8.1 DDoS & Telephony Scrubbing Middleware (Python/FastAPI)

```
import time
from fastapi import Request, HTTPException

IP_REQUEST_TRACKER = {}
BOMBING_THRESHOLD = 15 # Max auth requests per minute

@app.middleware("http")
async def rate_limit_middleware(request: Request, call_next):
    client_ip = request.client.host
    current_time = time.time()

    if client_ip not in IP_REQUEST_TRACKER:
        IP_REQUEST_TRACKER[client_ip] = []
```

```

# Sliding window: keep only requests from the last 60 seconds
IP_REQUEST_TRACKER[client_ip] = [t for t in IP_REQUEST_TRACKER[
client_ip] if current_time - t < 60]
IP_REQUEST_TRACKER[client_ip].append(current_time)

if len(IP_REQUEST_TRACKER[client_ip]) > BOMBING_THRESHOLD:
    raise HTTPException(status_code=429, detail="Volumetric attack
detected. Traffic dropped.")

return await call_next(request)

```

8.2 Deterministic Feature Extractor (Python)

```

import tldextract
import Levenshtein

HIGH_RISK_TLDS = [".xyz", ".top", ".info", ".buzz"]
TARGET_BRANDS = ["paypal", "microsoft", "google", "wellsfargo"]

def run_url_risk_rules(url: str):
    evidence = []
    extracted = tldextract.extract(url)
    domain = extracted.domain.lower()

    # Check 1: Typosquatting
    for brand in TARGET_BRANDS:
        if 0 < Levenshtein.distance(domain, brand) <= 2:
            evidence.append({"indicator": "Typosquatting", "detail": f"
Similar to {brand}"})

    # Check 2: Malicious Infrastructure
    if f"{extracted.suffix.lower()}" in HIGH_RISK_TLDS:
        evidence.append({"indicator": "High-Risk TLD", "detail": extracted
.suffix})

    return evidence

```